國立臺灣科技大學

資訊管理系 EMBA

碩士學位論文

網頁應用程式自動化生成方法與實作:以逆向工程生成軟體規格書為例

A Framework for Reverse Engineering Web Applications to Create Software Specifications and Drive AI Agent Automation

研　究　生　：　蕭清山

學　　號　：　M11209906

指 導 教 授 ：　黃政嘉 博士

中 華 民 國 一 百 一 十 四 年 五 月

M11209906

碩士學位論文指導教授推薦書

Master's Thesis Recommendation Form

系所： Department/Graduate Institute	資訊管理系 Department of Information Management
姓名： Name	蕭清山 HSIAO, CHING-SHAN
論文題目： (Thesis Title)	網頁應用程式自動化生成方法與實作:以逆向工程生成軟體規格書為例 A Framework for Reverse Engineering Web Applications to Create Software Specifications and Drive AI Agent Automation

係由本人指導撰述，同意提付審查。

This is to certify that the thesis submitted by the student named above, has been written under my supervision. I hereby approve this thesis to be applied for examination.

指導教授簽章：
Advisor's Signature

共同指導教授簽章（如有）：
Co-advisor's Signature (if any)

日期：
Date(yyyy/mm/dd) 2025 / 05 / 06

M11209906

碩士學位考試委員審定書

Qualification Form by Master's Degree Examination Committee

系所： Department/Graduate Institute	資訊管理系 Department of Information Management
姓名： Name	蕭清山 HSIAO, CHING-SHAN
論文題目： (Thesis Title)	網頁應用程式自動化生成方法與實作:以逆向工程生成軟體規格書為例 A Framework for Reverse Engineering Web Applications to Create Software Specifications and Drive AI Agent Automation

經本委員會審定通過，特此證明。

This is to certify that the thesis submitted by the student named above, is qualified and approved by the Examination Committee.

學位考試委員會

Degree Examination Committee

委員簽章：

Member's Signatures

羅乃維　　吳政嘉

張智傑

指導教授簽章：

Advisor's Signature

吳政嘉

共同指導教授簽章（如有）：

Co-advisor's Signature (if any)

系所（學程）主任（所長）簽章：

Department/Study Program/Graduate Institute Chair's Signature

日期：

Date(yyyy/mm/dd)

2025 / 05 / 16

摘要

本論文提出一個用於逆向工程網頁應用程式的框架，旨在產出可驅動 AI 代理執行自動化任務的軟體功能規格。針對企業應用程式間的互通性挑戰，提出一套無需存取原始碼的擷取方法，透過 HTTP 記錄（HTTP Archive）分析、參數因果關係分析、依賴關係圖構建與規格產生等技術來實現。其關鍵創新在於整合大型語言模型，能從這些規格中自動產生功能性的自動化腳本，並在多個 AI 平台上進行測試以驗證其可行性。該框架展望未來網頁應用程式將內建可供 AI 理解的機器可讀規格，讓整合工作能不依賴傳統 API 即可實現無縫串接。多個案例研究也驗證了此方法在無法直接存取 API 的情境下，仍可有效實現企業任務的自動化。

Abstract

This thesis presents a framework for reverse engineering web applications to create software functional specifications that drive AI agent automation. The research addresses interoperability challenges between diverse enterprise applications by developing a methodology that extracts interaction patterns without source code access, using HTTP Archive analysis, parameter causation analysis, dependency graph construction, and specification generation. A key innovation is the integration with large language models to automatically generate functional automation scripts from these specifications, with tests across multiple AI platforms demonstrating viability. The framework envisions a future where applications include machine-readable specifications for AI consumption, enabling seamless integration without traditional APIs, as validated through case studies that show effectiveness in automating enterprise tasks where direct API access is unavailable.

keywords Reverse engineering; web applications; software specifications; AI automation; HTTP Archive analysis.

Acknowledgments

I wish to extend my heartfelt gratitude to my advisor, Professor Jheng-Jia Huang, whose guidance, wisdom, and unwavering patience have shaped not only this thesis but also my personal growth as a researcher. Thank you for believing in me and pushing me to reach my full potential.

My sincere appreciation also goes to my committee members, Professor Nai-Wei Lo and Professor Chih-Chieh Chang, for their insightful comments and encouragement, which have greatly enhanced my work.

I am grateful to my fellow classmates from the 112 Information Management EMBA program and Team 2, whose camaraderie, humor, and encouragement brought joy and motivation during the most challenging moments of this journey.

To my loving wife, Linda, words cannot fully express my gratitude for your patience, love, and understanding. Your unwavering support and constant encouragement have been my foundation and inspiration.

Finally, my deepest thanks to my family, whose love, encouragement, and endless faith in me have carried me through this journey. Your support has been my greatest blessing.

Hsiao, Ching-Shan 2025/05/16

Table of Contents

摘要 III

Abstract IV

Acknowledgments V

Table of Contents VI

List of Figures IX

List of Tables XI

Chapter 1 Introduction 1

1.1 The Emerging Role of AI Agents 1

1.2 Research Gap and Opportunity 2

1.3 Research Objectives 3

1.4 Significance and Future Implications 3

1.5 Thesis Structure 4

Chapter 2 Related Work 5

2.1 Web Application Reverse Engineering 5

2.2 Parameter Extraction and Dynamic Analysis 6

2.3 Enterprise Application Integration and Interoperability 6

2.4 Specification Generation for Automation 6

2.5 Large Language Models in Software Engineering 7

2.6 Limitations in Current Approaches 7

2.7 Synthesis and Research Contribution 7

Chapter 3 Methodology 9

3.1 Data Structures and Node Representation 11
3.1.1 Node-Based HTTP Interaction Model 12
3.1.2 Parameter Origin Classification System 12
3.1.3 Recipe-Based Parameter Extraction 13
3.2 HAR File Filtering and Preprocessing 14
3.2.1 Single and Comparative Analysis Modes 16
3.2.2 Content-Based Filtering Algorithm 16
3.2.3 Redirect Chain Processing 17
3.2.4 Comparative Analysis Algorithm 18
3.2.5 Design Rationale and Significance 18
3.3 Causation Analysis and Parameter Extraction 19
3.3.1 Causation versus Correlation 20
3.3.2 Recipe-Based Parameter Classification System 20
3.3.3 Pattern Detection Algorithm 23
3.3.4 DOM Traversal and Selection Engine 24
3.3.5 Depth-Limited Search Optimization 24
3.3.6 User Input Identification Heuristics 25
3.3.7 Design Rationale and Significance 25
3.4 Dependency Graph Construction 26
3.4.1 Execution Order Determination 28
3.4.2 Visualization Generation 29
3.4.3 User Input Requirements Detection 29
3.5 Specification Generation 30
3.5.1 Template-Based Specification System 30

3.5.2 Parameter Specification Generation 31
3.5.3 Helper Function Documentation 33
3.5.4 Design Rationale and Significance 33
3.6 AI-Based Script Generation 34
3.6.1 LLM Integration Architecture 35
3.6.2 Sophisticated Prompt Engineering 36
3.6.3 Future Extensions 38
Chapter 4 Results and Discussion 39
4.1 Case Study Implementation 39
4.2 Analysis of Framework Performance 43
4.3 Integrated Evaluation and Implications 45
4.4 Limitations and Future Research 46
Chapter 5 Conclusion 47
References 48

List of Figures

Figure 1: Framework Pipeline Stages with Binary Flag Control 9

Figure 2: Processing pipeline for Single and Compare HAR analysis modes. . 15

Figure 3: Pseudocode for parameter stability classification based on HAR entry comparison. 18

Figure 4: Decision flow for classifying parameters into recipe-based extraction types. 22

Figure 5: Pseudocode for detecting parameter patterns and assigning recipe types 23

Figure 6: Depth limits for query and POST parameter searches. 24

Figure 7: Parameter Causation Between Requests 27

Figure 8: Pseudocode for Execution Order Determination 28

Figure 9: Example of Execution Graph 29

Figure 10: Recipe Template Sample 31

Figure 11: Example of Dynamic Parameter Specification 32

Figure 12: Abstract Class for AI Script Generator 36

Figure 13: moodle2.ntust.edu.tw: All Nodes 41

Figure 14: moodle2.ntust.edu.tw: Dependent Nodes Only 41

Figure 15: tip.railway.gov.tw: All Nodes 42

Figure 16: tip.railway.gov.tw: User Input File (user_input.json) 42

Figure 17: mvdis.gov.tw: All Nodes 42

Figure 18: camping.wuling-farm.com.tw: All Nodes 43

List of Tables

Table 1: Sourcing Model for Parameter Origin Classification..... 13

Table 2: Definition of Recipe Types for Parameter Extraction..... 20

Table 3: Summary of Case Study Applications and Results..... 40

Table 4: Challenging Parameters Encountered by Application 44

Chapter 1 Introduction

Enterprise environments today are characterized by the extensive use of diverse web applications, including ERP, CRM, standalone accounting software, HR management systems, and POS systems. A common challenge is that these applications, particularly those developed in different eras or by distinct vendors, lack interoperability. In such cases, operational staff often rely heavily on spreadsheet software to bridge gaps between systems, manually copying and pasting data from exported spreadsheets into various web applications. This repetitive and error-prone process is time-consuming and resource-intensive.

Automating the extraction of data from spreadsheet files and its subsequent entry into web applications could significantly improve operational efficiency. However, many enterprise web applications are externally developed and lack accessible APIs for direct integration. This creates a technical barrier that has traditionally been difficult to overcome without substantial custom development efforts or expensive integration platforms.

1.1 The Emerging Role of AI Agents

Recent advancements in artificial intelligence, particularly in large language models (LLMs), present a novel approach to this longstanding challenge. These models demonstrate remarkable capabilities in understanding and generating code, as well as

interpreting structured specifications. As AI continues to evolve, we are witnessing the emergence of AI agents capable of performing complex tasks with minimal human intervention.

The concept of AI agents autonomously interacting with web applications represents a paradigm shift in enterprise automation. Rather than relying on traditional UI-based automation or API integration, these agents could potentially understand the underlying structure and requirements of web applications directly from machine-readable specifications. This approach bypasses many traditional integration challenges by allowing the AI to dynamically generate and execute the necessary interaction scripts.

1.2 Research Gap and Opportunity

Despite the potential of LLMs and AI agents, there remains a significant gap in how these technologies can be systematically applied to web application automation. Current approaches often involve either:

1. Browser-based automation tools that simulate user interactions but are fragile and maintenance-intensive
2. Browser-based automation tools are inefficient due to the need to fully render user interfaces and execute JavaScript
3. Custom-developed integration solutions that require significant development resources
4. Ad-hoc use of AI for generating scripts without a structured methodology or framework

What's missing is a comprehensive framework that bridges the gap between existing web applications and the capabilities of modern AI systems. Such a framework would need to systematically reverse engineer web applications, generate structured specifications, and enable AI agents to interpret these specifications to automate workflows without human intervention.

1.3 Research Objectives

This thesis aims to develop and validate a framework for reverse engineering web applications to create software specifications that can drive AI agent automation. Specifically, the research objectives are to:

1. Design a methodology for systematically extracting the interaction patterns and data requirements from web applications without access to source code
2. Develop a structured specification format that captures the essential elements of web application interactions in a machine-readable format
3. Demonstrate how modern LLMs can interpret these specifications to generate functional automation scripts
4. Validate the effectiveness of the framework through practical web applications

1.4 Significance and Future Implications

The significance of this research extends beyond immediate automation benefits. As AI agents become more sophisticated, they will increasingly require standardized ways to understand and interact with existing software systems. This research envisions a future where enterprise web applications include dedicated sections with specifications intended not for human users but for AI agents.

Such a future would enable seamless integration between diverse applications without traditional API development, dramatically reducing the manual effort currently required for cross-system workflows. Additionally, it would provide a transition path for legacy systems to participate in modern, AI-driven automation ecosystems without requiring fundamental redesign.

1.5 Thesis Structure

The remainder of this thesis is organized as follows:

- Chapter 2 reviews relevant literature and related work on web automation, reverse engineering techniques, and the application of AI to software integration challenges
- Chapter 3 presents the detailed methodology of the proposed framework, including data structures, HAR file processing, causation analysis, dependency graph construction, specification generation, and AI-based script generation
- Chapter 4 describes the implementation and evaluation of the framework through case studies
- Chapter 5 discusses the results, limitations, and implications of the research, followed by conclusions and directions for future work

This research contributes to the emerging field of AI-augmented software engineering and provides a practical approach to addressing the persistent challenge of enterprise application integration in an era of rapidly evolving AI capabilities.

Chapter 2 Related Work

The automation of web applications and the generation of machine-readable specifications from black-box systems intersect multiple research domains, including web reverse engineering, robotic process automation (RPA), enterprise application integration (EAI), and the application of large language models (LLMs) in software engineering.

2.1 Web Application Reverse Engineering

Reverse engineering of web applications has evolved significantly since the early reliance on static HTML analysis. Tools like WebScarab and Burp Suite introduced traffic inspection as a means to analyze application behavior without source code access. Research by Felmetsger et al. [1] demonstrated how traffic patterns could uncover logic vulnerabilities, paving the way for structural analysis of dynamic parameters. More recently, Mesbah et al. [2] proposed DOM state tracking to analyze Ajax-based applications, highlighting the complexity of client-side interactions.

The use of HAR (HTTP Archive) files has since become a de facto standard for capturing and analyzing HTTP-level behavior [3]. These artifacts allow inspection of sequential request-response patterns, which are instrumental in uncovering causation relationships between dynamic parameters. Approaches such as those by Păsăreanu et al. [4] integrated model inference from request flows to reconstruct user workflows and dependency chains.

2.2 Parameter Extraction and Dynamic Analysis

Understanding how parameters evolve across requests is critical for automation. Traditional approaches relied on DOM scraping and brittle XPath rules [5]. More recent work by Minamide [6] on static analysis of HTML and JavaScript, as well as dynamic taint analysis techniques [7], have improved accuracy. The "recipe-based" approach, introduced in this thesis, builds upon these ideas by abstracting extraction patterns into generalized templates, allowing AI systems to interpret and regenerate such logic across applications.

2.3 Enterprise Application Integration and Interoperability

Enterprise application integration remains a longstanding challenge. Linthicum [8] categorized integration strategies, from direct APIs to message-oriented middleware. However, many legacy applications lack APIs, forcing integration through UI automation. Robotic Process Automation (RPA) addresses this through tools like UiPath and Automation Anywhere [9][10], but these are often brittle and require extensive maintenance. This thesis distinguishes itself by leveraging causality analysis and dependency graph construction to create stable automation workflows.

2.4 Specification Generation for Automation

The idea of machine-readable specifications is not new. The WSDL and OpenAPI initiatives sought to document APIs [11][12], while DSLs (Domain-Specific Languages) like Gherkin in BDD frameworks attempted to document expected system behavior [13]. However, none directly addressed specification creation from opaque

systems. The use of structured recipes to generate YAML-based specifications, as proposed here, introduces a novel bridge between interaction capture and AI-consumable logic.

2.5 Large Language Models in Software Engineering

The rise of LLMs like Codex [14], Code Llama [15], and Gemini [16] introduced new possibilities in automation. They can now generate executable code, interpret instructions, and complete tasks across languages and frameworks. Frameworks like ReAct [17] and Toolformer [18] demonstrate that LLMs can reason through workflows when provided with sufficient structure. This thesis builds on this foundation by providing specifications that serve as interpretable plans for these models.

2.6 Limitations in Current Approaches

Despite progress, existing tools struggle with generalization across applications. TestRigor [19] and Selenium IDE [20] automate workflows but fail in sessions with dynamically generated tokens or complex redirect chains. Few systems attempt to extract causation chains automatically or separate user input from application-generated data—two challenges directly addressed in this research through recipe abstraction and user input heuristics.

2.7 Synthesis and Research Contribution

By combining traffic analysis, pattern classification, dependency modeling, and AI prompt design, this research fills a gap in automating workflows for enterprise

systems lacking APIs. The proposed framework stands apart in that it not only extracts the logic of web applications but produces AI-readable specifications enabling end-to-end automation.

Chapter 3 Methodology

The methodology presented in this thesis establishes a systematic framework for reverse engineering web applications through analysis of HTTP interactions captured in HAR (HTTP Archive) files. Rather than focusing on the UI layer, this approach works directly with network traffic to reconstruct the underlying logic and data flow of the application.

The framework operates through a configurable pipeline with five core stages, each implemented as a binary flag (see Figure 1) for flexible execution control:

```
STEP_FILTER_HAR     = 0b00001      # 1: filter HAR file
STEP_ANALYZE_HAR    = 0b00010      # 2: analyze HAR and create nodes
STEP_CREATE_GRAPH   = 0b00100      # 4: create dependency graph
STEP_GENERATE_SPEC  = 0b01000      # 8: generate specification
STEP_GENERATE_SCRIPT = 0b10000     # 16: generate script from spec
```

Figure 1: Framework Pipeline Stages with Binary Flag Control

These binary flags can be combined to execute specific subsets of the pipeline, allowing for flexible workflow configurations:

1. **HAR File Filtering**: The framework supports two distinct analysis modes - 'single' for processing individual HAR files and 'compare' for analyzing differences between two HAR files. The comparison mode is particularly valuable for identifying session-dependent parameters by comparing multiple recordings of the same workflow. In this stage, the framework filters out irrelevant requests such as image, CSS, JavaScript resources, focusing

exclusively on HTML and JSON responses that contain dynamic content and potential data dependencies.

2. **Causation Analysis**: This stage implements a temporal causation analysis algorithm that examines the chronological sequence of HTTP requests and responses to identify causal relationships. Unlike simple correlation or dependency analysis, the framework specifically searches for values in earlier responses that appear in later requests, establishing true causation chains. The system employs a sophisticated recipe-based pattern recognition system that can detect parameters embedded in various contexts within HTML and JSON responses.
3. **Dependency Graph Generation**: The causation relationships discovered in the previous stage are represented as a directed graph where nodes represent HTTP requests and edges represent data dependencies. This graph visualization enables clear identification of execution order dependencies and provides a debugging mechanism for understanding complex web application flows. The graph generation process employs topological sorting to determine a valid execution sequence that respects all dependencies, ensuring that every request has access to all required parameter values before execution.
4. **Specification Generation**: Utilizing a template-based approach, the framework converts the dependency graph into a structured specification document based on the "recipe" templates. These templates provide a standardized format that is both human-readable and well-structured for consumption by AI models. The specification deliberately uses natural language patterns and explicit code

examples that AI language models can interpret accurately. This format is designed to be language-agnostic, allowing for generation of automation scripts in Python, Node.js, Go, or other programming languages, providing flexibility for different runtime environments.

5. **Script Generation**: The final stage employs a carefully crafted prompt engineering approach to guide AI models (supporting both OpenAI and Anthropic models) in transforming the specification into executable code. The prompt design focuses on producing robust, production-ready code with proper error handling, parameter extraction, and session management. The system supports different AI models and configurations, allowing for selection of the most appropriate model based on the complexity of the specification.

The integration of these stages creates a comprehensive reverse engineering pipeline that can automatically discover and model complex web application behaviors without requiring manual interpretation of network traffic or HTML structures. The methodology's strength lies in its ability to automatically reconstruct the underlying logic of web applications through causal analysis of network interactions, making it particularly suitable for applications with complex session management, CSRF protection, and dynamic parameter generation.

3.1 Data Structures and Node Representation

The foundation of the reverse engineering framework lies in its ability to represent and model complex HTTP interactions in a structured, programmatically accessible format. This section introduces the core data structures that enable the precise modeling of web application behavior.

3.1.1 Node-Based HTTP Interaction Model

At the heart of the framework is a node-based representation system that models each HTTP request as a discrete node with defined properties and relationships. This approach transforms the linear sequence of HTTP transactions into a structured graph that reveals the underlying application logic. The framework implements a two-tier abstraction model:

1. **Node Collection**: A container that manages all HTTP nodes and their relationships
2. **Individual Nodes**: Discrete entities representing specific HTTP requests

Each node encapsulates both the structural properties of the request (URL, method) and the semantic relationships between requests (where parameters originate from). This dual representation allows the framework to capture not just the mechanics of each request but also the logical connections between them. The node model separates parameters into two distinct categories:

- Query parameters: Values passed in the URL
- POST parameters: Values sent in the request body

This separation reflects the different handling mechanisms typically employed by web applications and enables specialized processing for each type.

3.1.2 Parameter Origin Classification System

A key innovation in the data structure design is the parameter origin classification system. Rather than treating all parameters as equal, the framework implements a sophisticated sourcing model that categorizes each parameter based on its origin:

Table 1: Sourcing Model for Parameter Origin Classification

Source ID	Description	Significance
-2	Static value	Parameter is constant across sessions
-1	User input value	Parameter requires external input
0+	Response-derived value	Parameter extracted from previous response

This classification fundamentally transforms how the automation process works. Rather than simply replaying captured values (which would fail for session tokens and other dynamic values), the framework models how to obtain each value during execution. This approach makes the resulting automation resilient to session changes, time-based tokens, and other dynamic elements.

3.1.3 Recipe-Based Parameter Extraction

The most significant contribution of the data structure design is its recipe-based parameter extraction system. Each parameter is linked to a "recipe" that defines precisely how to extract its value from a source response. This system includes:

- **Source node reference**: Which previous response contains the needed value
- **Extraction selector**: DOM selectors for locating elements in HTML
- **Processing method**: How to extract and transform the raw value

The recipe concept abstracts the extraction logic, creating a declarative rather than imperative approach to parameter handling. This design choice enables the

framework to handle a wide range of parameter extraction scenarios without requiring custom code for each case.

For example, a CSRF token might be represented with a recipe indicating that it should be extracted from a hidden form field in the response of Node 4, while a session ID might be extracted from a JavaScript variable assignment in Node 2. This abstraction layer is what enables the framework to automatically generate automation code that correctly handles complex session management without requiring manual intervention.

3.2 HAR File Filtering and Preprocessing

The initial stage of the framework focuses on processing HTTP Archive (HAR) files to extract relevant request-response pairs and prepare them for deeper analysis. HAR files contain comprehensive records of all browser-server interactions, including resources like images, stylesheets, and scripts that are not directly relevant to application logic. This section describes the methodology employed to filter and preprocess these files.

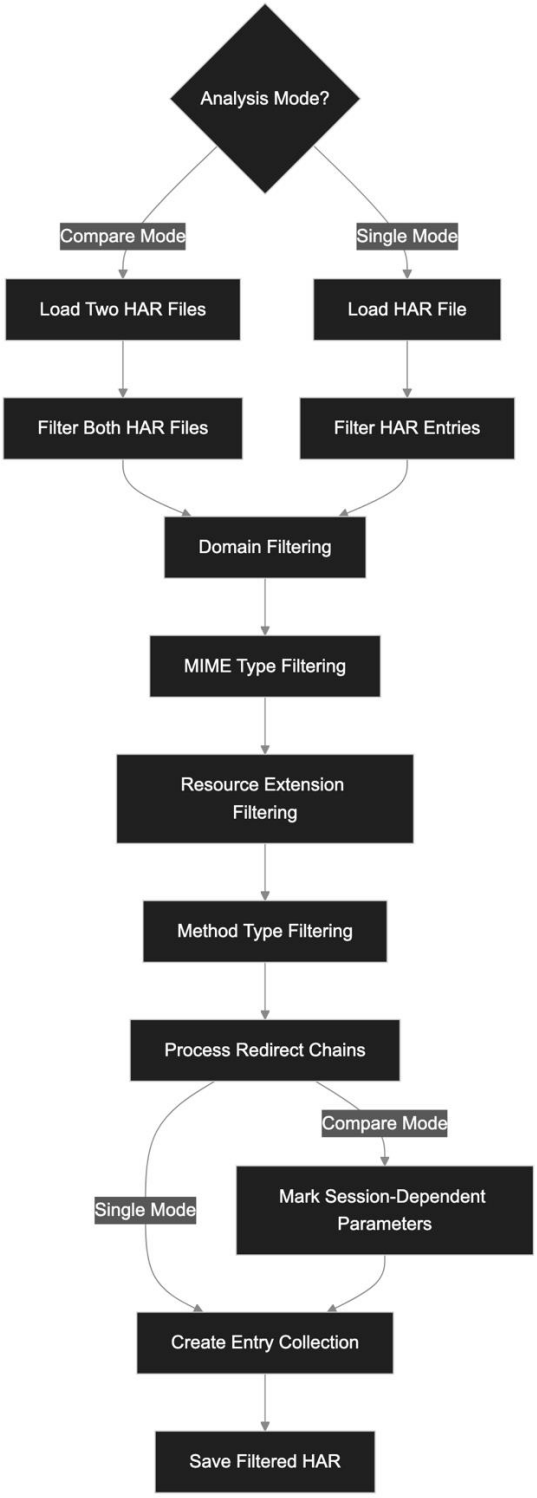

Figure 2: Processing pipeline for Single and Compare HAR analysis modes.

3.2.1 Single and Comparative Analysis Modes

The framework supports two distinct operational modes for HAR processing (see Figure 2 for the full pipeline):

1. **Single HAR Mode**: Processes a single recording of a web application workflow, extracting the core request-response patterns.
2. **Comparison Mode**: Analyzes two separate recordings of the same workflow, identifying which parameters are session-dependent (changing between sessions) and which are static.

The comparison mode represents a significant methodological advancement over traditional approaches. By comparing two instances of the same workflow, the framework can automatically determine parameter stability without requiring manual analysis. This capability is particularly valuable for applications where distinguishing between session tokens and fixed application parameters is critical for reliable automation.

3.2.2 Content-Based Filtering Algorithm

The filtering algorithm applies multiple layers of refinement to extract only the HTTP transactions that contribute to application logic:

1. **Domain Filtering**: Restricts analysis to a single domain to eliminate third-party content
2. **MIME Type Filtering**: Selects only content types that typically contain application logic:
    - text/html: Web pages with DOM structure
    - application/json: Structured data responses

- application/xml: Structured XML data
- text/plain: Potential data responses

3. **Resource Extension Filtering**: Eliminates static resources by extension pattern
4. **Method Filtering**: Focuses on GET and POST requests with specific content types

The filtering significantly reduces the volume of data processed in subsequent stages, focusing computational resources on transactions that contain application logic rather than static resources.

3.2.3 Redirect Chain Processing

A key preprocessing challenge involves correctly handling HTTP redirects. Web applications frequently use redirects for authentication flows, which creates discontinuities in the raw HAR timeline. The framework implements a sophisticated redirect chain processing algorithm that:

1. Identifies responses with 3XX status codes
2. Traces subsequent requests to the redirect targets
3. Reconstructs the logical request chains across redirects
4. Updates responses to contain the final destination content

This redirect processing ensures that dependent parameters can be correctly extracted even when they span redirect chains, a common pattern in authentication and session management flows.

3.2.4 Comparative Analysis Algorithm

In comparison mode, the framework implements a differential analysis algorithm to automatically classify parameter stability (see Figure 3 for the pseudocode). This algorithm enables the framework to make intelligent decisions about parameter handling without requiring manual intervention. Parameters marked as static can be treated as constants in the generated automation script, while dynamic parameters require extraction from responses or user input.

```
// Pseudocode for parameter stability classification
function compareHarFiles(targetEntries, referenceEntries):
    for each entry in targetEntries:
        find matching entry in referenceEntries with same URL path

        for each query parameter in entry:
            if parameter exists in matching entry:
                if values are identical:
                    mark parameter as static (session_dependent = 0)
                else:
                    mark parameter as dynamic (session_dependent = 1)

        for each POST parameter in entry:
            if parameter exists in matching entry:
                if values are identical:
                    mark parameter as static (session_dependent = 0)
                else:
                    mark parameter as dynamic (session_dependent = 1)

    return targetEntries with stability classifications
```

Figure 3: Pseudocode for parameter stability classification based on HAR entry comparison.

3.2.5 Design Rationale and Significance

The filtering and preprocessing methodology was designed with several key objectives:

1. **Noise Reduction**: HAR files can contain thousands of requests, many irrelevant to application logic. Filtering reduces this to the essential transactions.
2. **Cross-Session Analysis**: The comparison mode enables automatic discovery of session dynamics without requiring application-specific knowledge.
3. **Logical Continuity**: The redirect processing ensures that logical user flows remain intact despite HTTP protocol discontinuities.

The filtering and preprocessing stage transforms raw network captures into coherent, logically structured request-response sequences that represent the application's core behavior. This transformation is foundational to the causation analysis that follows, enabling the framework to focus computational resources on transactions that contribute to the application's business logic.

3.3 Causation Analysis and Parameter Extraction

The causation analysis stage represents the core intelligence of the framework, where temporal sequence of HTTP interactions is transformed into a logical causation model. This process discovers how parameters in requests derive their values from previous responses, establishing true causation chains that form the foundation for automated script generation.

3.3.1 Causation versus Correlation

In web application analysis, distinguishing causation from mere correlation is crucial. The framework implements a temporal causation discovery algorithm that assumes potential causation when:

1. Parameter value X appears in response A
2. The same value X appears in a subsequent request B
3. Request B occurs chronologically after response A

This temporal sequence establishes potential causation pathways, which are then refined through pattern analysis to confirm actual causation relationships.

3.3.2 Recipe-Based Parameter Classification System

The core innovation in the causation analysis is the recipe-based classification system (see Figure 4 for the decision flow). The framework categorizes parameter extraction methods into distinct recipe types, each representing a specific extraction context:

Table 2: Definition of Recipe Types for Parameter Extraction

R#	Description	Pattern Example	Context
1	Static value	Direct assignment	Constants or hardcoded values
2	Text content	Element inner text	Values in HTML content
3	JS assignment	var = 'value';	JavaScript variable assignments
4	Query in JS	window.location='example.com?key=value';	URL parameters in JavaScript
5	Query in text	href="page?token=abc123"	URL parameters in attributes
6	Function args	submitForm('token123', userId);	Function call arguments
7	Query in function	openPage('login?token=abc123');	URL in function arguments
8	User input	username/password fields	Interactive input requirements
9	JSON path	{"data":{"token":"abc123"}}	Values in structured JSON

This classification transforms the parameter extraction problem from an unstructured text analysis challenge into a structured pattern recognition task. Each recipe type has specific detection and extraction algorithms tailored to its context.

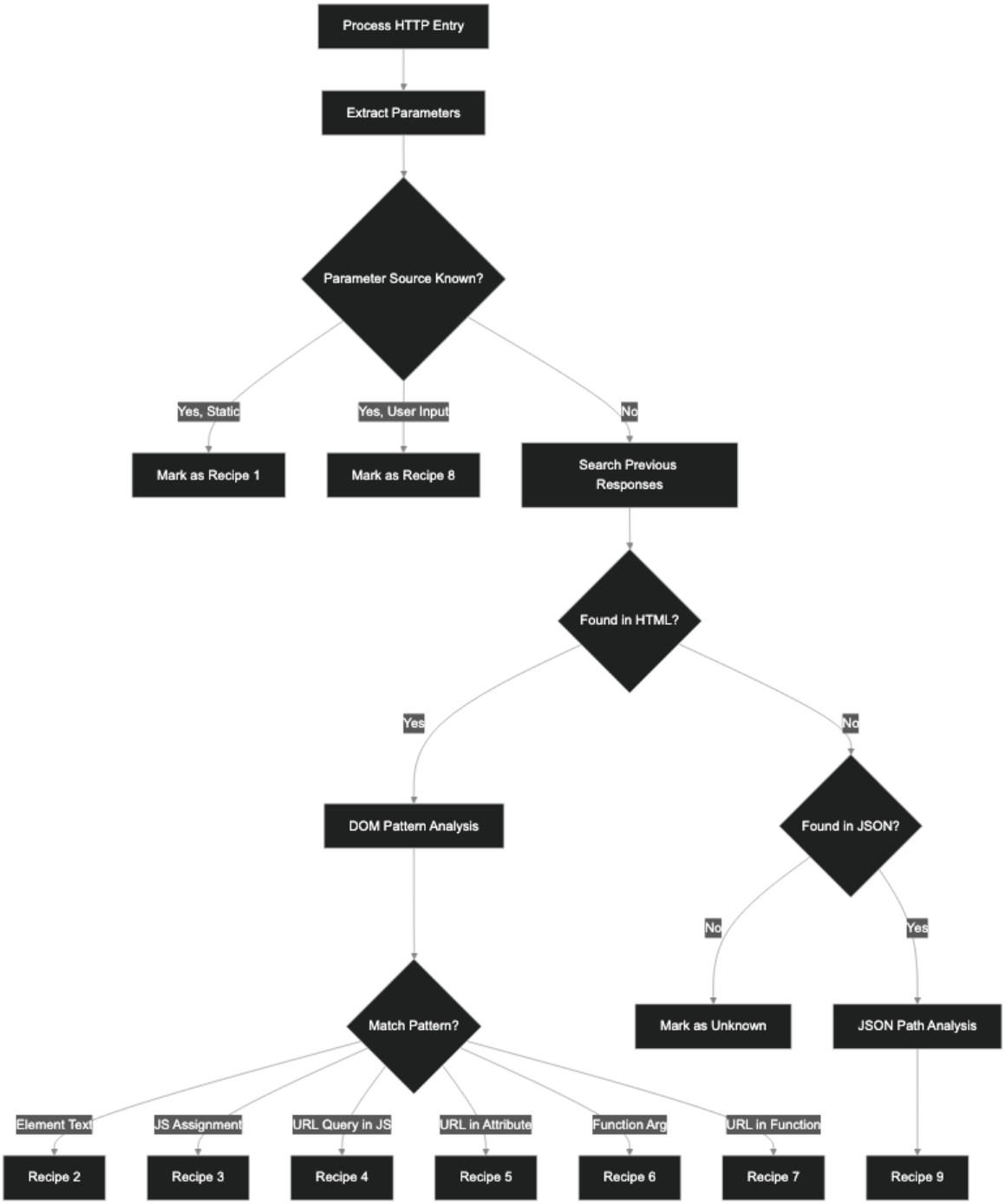

Figure 4: Decision flow for classifying parameters into recipe-based extraction types.

3.3.3 Pattern Detection Algorithm

The framework implements a sophisticated pattern detection system to identify the correct recipe for each parameter value (see Figure 5 for the pseudocode). The core algorithm is outlined in the following pseudocode:

```
// Pseudocode for the causation analysis algorithm
function findParameterSource(paramName, paramValue, currentNodeIndex):
    // Check if exact value exists in user input
    if valueExistsInUserInput(paramValue):
        return createRecipe(8, userInputKey)

    // Search backwards through previous responses
    for prevNodeIndex = currentNodeIndex-1 down to 0:
        prevResponse = getResponseText(prevNodeIndex)

        // Find potential DOM elements containing the value
        matchedElements = findMatchingElements(prevResponse, paramName, paramValue)

        for element in matchedElements:
            // Try different extraction patterns
            if elementContainsExactValue(element, paramValue):
                return createRecipe(2, element)

            if containsJsAssignment(element, paramValue):
                return createRecipe(3, element)

            if containsQueryInJs(element, paramValue):
                return createRecipe(4, element, extractQueryKey())

            // Additional pattern checks for recipes 5-7...

        // Check for JSON content
        if isJsonResponse(prevResponse):
            jsonPaths = findJsonValues(parseJson(prevResponse), paramValue)
            if jsonPaths:
                return createRecipe(9, jsonPaths[0])

    // No source found
    return null
```

Figure 5: Pseudocode for detecting parameter patterns and assigning recipe types

The algorithm applies multiple pattern recognition techniques, starting with simple matches and progressively trying more complex patterns. This approach enables the framework to handle a wide range of parameter embedding techniques without requiring application-specific customization.

3.3.4 DOM Traversal and Selection Engine

For HTML responses, the framework employs a specialized DOM traversal engine based on BeautifulSoup that:

1. Identifies elements containing target parameter values
2. Analyzes the context of the value (attribute, text content, etc.)
3. Generates precise CSS selectors for future extraction
4. Tests selector specificity to ensure uniqueness

The selector generation is particularly sophisticated, creating the minimum selector required to uniquely identify the target element. This minimizes fragility in the resulting automation script, enhancing robustness against minor DOM changes.

3.3.5 Depth-Limited Search Optimization

Enterprise applications often involve large response payloads, which could make exhaustive searching computationally expensive. The framework implements depth-limited search with context-aware thresholds (see Figure 6 for parameter search limits):

```
GET_SEARCH_DEPTH = 5    // Limit for query parameter search
POST_SEARCH_DEPTH = 30  // Deeper limit for POST data search
```

Figure 6: Depth limits for query and POST parameter searches.

This optimization recognizes that POST parameters typically require deeper search due to form submissions, while query parameters often derive from more recent responses. The dual-threshold approach balances computational efficiency with thorough causation discovery.

3.3.6 User Input Identification Heuristics

Distinguishing between application-generated values and user-provided inputs is critical for creating usable automation scripts. The framework implements several heuristics for identifying likely user inputs:

1. Small numeric values within thresholds might be user selections
2. Empty string values in POST data often represent user inputs
3. Field names matching common patterns (username, password, etc.)
4. Parameters lacking discovery patterns in previous responses

These heuristics help identify interaction points requiring external input, which are marked with Recipe 8 for appropriate handling in the generated script.

3.3.7 Design Rationale and Significance

The causation analysis methodology represents a significant advancement over traditional web scraping approaches that rely on fixed selectors or exact replay of captured values. Key advantages include:

1. **Adaptation to Dynamic Content**: By discovering how values are generated rather than hardcoding them, the framework can adapt to session-specific changes.

2. **Reduced Brittleness**: The recipe-based approach focuses on extraction patterns rather than specific DOM paths, increasing resilience to UI changes.
3. **Self-Documenting Logic**: Each parameter's recipe explicitly documents its source and extraction method, creating inherently self-documenting specifications.
4. **Platform Independence**: The causation analysis operates at the HTTP level, making it independent of specific browser implementations or JavaScript engines.

The causation analysis transforms opaque HTTP sequences into a transparent, logical model of application behavior. This model enables not just replay of recorded actions but true reconstruction of the application's internal logic, allowing for robust automation across changing sessions and evolving application interfaces.

3.4 Dependency Graph Construction

The framework employs a directed multigraph structure to represent HTTP request dependencies (see Figure 7). Unlike a simple directed graph, a multigraph can maintain multiple distinct edges between the same nodes, which is essential for representing different parameter dependencies between the same request pairs.

The graph is constructed with the following elements:

- **Nodes**: HTTP requests with their base URLs and parameter collections
- **Edges:** Parameter dependencies between requests
- **Edge Properties**: Recipe information and parameter mapping details

This structure enables precise modeling of complex dependencies where multiple parameters in a request may derive from different source responses through different extraction methods.

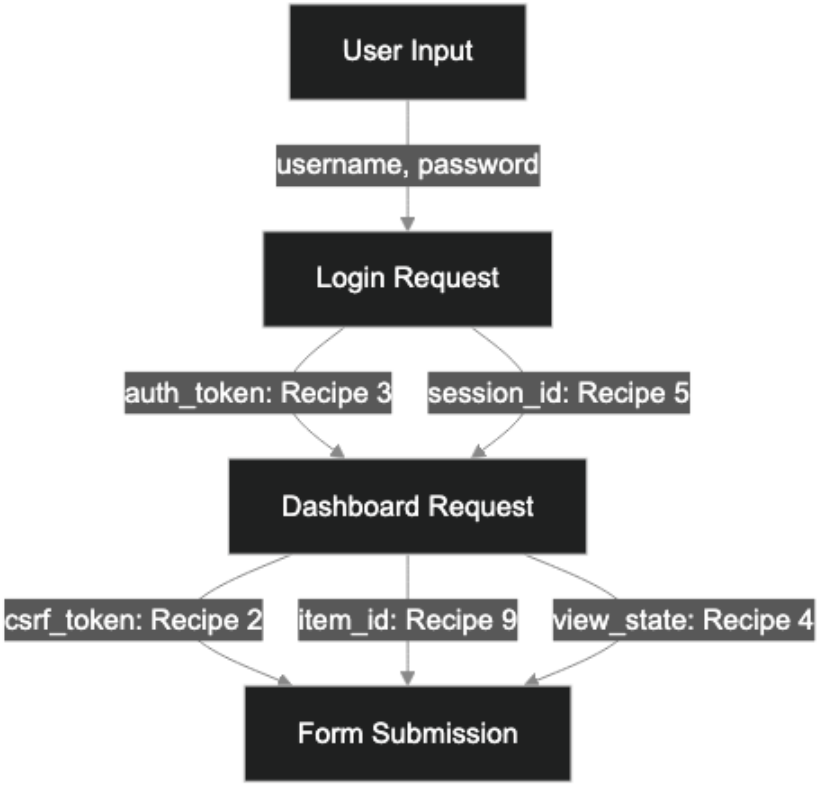

Figure 7: Parameter Causation Between Requests

3.4.1 Execution Order Determination

A critical function of the dependency graph is determining a valid execution order that respects all parameter dependencies. The framework implements a topological sorting algorithm with special handling for virtual nodes:

This algorithm (see Figure 8) ensures that every request in the execution chain has access to all required parameter values before execution, creating a valid sequence that can be translated into procedural code.

```
// Pseudocode for execution order determination
function getDependencyChain(targetNode = null):
    if targetNode is null:
        targetNode = highestNodeId  // Default to last node

    // Get all nodes that targetNode depends on (ancestors)
    dependencies = {targetNode}
    userInputDependencies = getAllUserInputDependentNodes()

    // Add user input dependencies and their ancestors
    for node in userInputDependencies:
        if node <= targetNode:  // Only include if not after target
            dependencies.add(node)
            dependencies.addAll(getAncestors(node))

    // Add direct ancestors of target node
    dependencies.addAll(getAncestors(targetNode))

    // Always include the initial request (node 0) if feasible
    if shouldIncludeInitialNode():
        dependencies.add(0)

    // Sort in ascending order (typically matches chronological order)
    return sortAscending(dependencies)
```

Figure 8: Pseudocode for Execution Order Determination

3.4.2 Visualization Generation

The framework generates two distinct graph visualizations to facilitate analysis and debugging (see Figure 9):

1. **Complete Graph**: Shows all nodes and dependencies, including those not in the execution chain
2. **Execution Graph**: Focuses only on nodes required for the target workflow

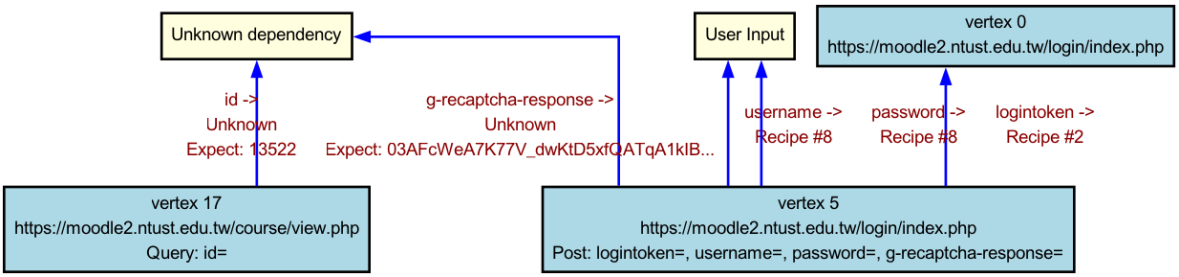

Figure 9: Example of Execution Graph

3.4.3 User Input Requirements Detection

The graph construction process automatically identifies nodes requiring user input by analyzing:

1. Parameters marked with Recipe 8 (user input)
2. Parameters with unknown sources
3. Parameters with patterns suggesting user interaction (form fields, etc.)

These nodes are connected to the virtual "User Input" node (-1) in the graph, creating clear documentation of interactive requirements in the automation workflow.

3.5 Specification Generation

After constructing the dependency graph, the framework transforms this structured representation into a detailed automation specification document. This specification serves as a bridge between the abstract graph model and concrete implementation code, providing a human-readable yet precisely structured format that can be consumed by AI models for script generation.

3.5.1 Template-Based Specification System

The framework employs a template-based approach for generating specifications, which offers several advantages over direct code generation:

1. Consistency in representation across different parameter types
2. Explicit documentation of extraction logic
3. Separation of logical structure from implementation details
4. Neutral format that can be translated to multiple programming languages

The template system uses YAML-based recipe definitions (see Figure 10) that standardize how each extraction method is represented in the specification document.

```
recipe_2:
  title: "Matched value in text content"
  template: |
    - {param_type}: {param_name} = '{param_value}' (Recipe 2)
      - Recipe Details:
        - Selector: node_{node_id}_selector
        - Recipe: node_{node_id}_recipe
      - Code Example:
        # the element value/text directly contains the desired value
        node_{node_id}_selector = {
            "tag": "{tag}",
            "name": "{name}",
            "type": {type},
            "id": "{id}",
            "class": {class_list},
            "matched_attr": {matched_attr}
        }
        node_{node_id}_recipe = "2 Matched value in text content"
        node_{node_id}_{param_container}["{param_name}"] = extract_value
        (recipe=node_{node_id}_recipe, selector=node_{node_id}_selector,
        response_text=node_{source_node}_response_text)

recipe_3:
  title: "Matched value in js assignment"
  template: |
    - {param_type}: {param_name} = '{param_value}' (Recipe 3)
```

Figure 10: Recipe Template Sample

3.5.2 Parameter Specification Generation

For each parameter, the framework generates a detailed specification (see Figure 11) using the appropriate template:

```
Node 3:
- Method: POST
- Base URL: https://example.com/api/submit
- Query Parameter: session_token = dynamically extracted using Recipe 5 from node_2_response_text
  - Recipe Details:
    - Selector: node_3_selector
    - Recipe: node_3_recipe
  - Code Example:
    # the element value/text contains a URL with query parameters
    node_3_selector = {
        "tag": "a",
        "name": "",
        "type": None,
        "id": "session-link",
        "class": ["auth-link"],
        "matched_attr": ["href"]
    }
    node_3_recipe = "5 Matched value in query which key=token in text content"
    node_3_query_string["session_token"] = extract_value(recipe=node_3_recipe, selector=node_3_selector,
- POST Data:
  - POST Data: csrf_token = dynamically extracted using Recipe 2 from node_2_response_text
    - Recipe Details:
      - Selector: node_3_post_selector
      - Recipe: node_3_post_recipe
    - Code Example:
      # the element value/text directly contains the desired value
      node_3_post_selector = {
          "tag": "input",
          "name": "csrf_token",
          "type": "hidden",
          "id": "csrf",
          "class": [],
          "matched_attr": ["value"]
      }
      node_3_post_recipe = "2 Matched value in text content"
      node_3_post_data["csrf_token"] = extract_value(recipe=node_3_post_recipe, selector=node_3_post_sel
  - POST Data: username = from user_input["username"] (Recipe 8)
  - POST Data: password = from user_input["password"] (Recipe 8)
- Response stored as node_3_response_text for dynamic extraction
```

Figure 11: Example of Dynamic Parameter Specification

This detailed specification provides:

1. **Clear Traceability**: Each parameter explicitly states its source
2. **Extraction Method**: The recipe type indicates how to obtain the value
3. **Selector Information**: Precise selectors for DOM-based extraction

4. **Code Example**: Pseudo-implementation showing how to extract the parameter
5. **Response Storage**: Indication of which responses need to be preserved for later use

The level of detail in the specification is deliberately designed to be sufficient for a language model to generate functional code without requiring additional context.

3.5.3 Helper Function Documentation

The specification includes a comprehensive set of helper function definitions that support the extraction recipes:

1. extract_value: The core function for applying recipes to extract parameters
2. fetch_html_element_value: For retrieving values from HTML using selectors
3. retrieve_query_value: For extracting query parameters from URLs
4. parse_js_assignment: For extracting values from JavaScript assignments
5. parse_js_function_call: For extracting values from function call arguments

These helper function definitions ensure that the generated script has all necessary utilities for implementing the extraction logic, regardless of the target programming language.

3.5.4 Design Rationale and Significance

The specification generation methodology bridges the critical gap between analysis and implementation through several innovative approaches:

1. **Language-Agnostic Format**: By using a structured yet language-neutral format, the specification can be translated to various programming languages.
2. **Self-Contained Logic**: Each parameter specification contains all information needed for its extraction, making the specification self-contained.
3. **Pseudo-Implementation**: The inclusion of code examples provides concrete guidance for implementation while still allowing flexibility in the final code structure.
4. **Explicit Execution Order**: The specification clearly documents the required execution sequence based on the dependency analysis.

The specification serves as both documentation and implementation guide, transforming the abstract dependency graph into concrete instructions that can be followed by human developers or AI code generators. This approach separates the logical understanding of the application from the implementation details, enabling more flexible and maintainable automation solutions.

3.6 AI-Based Script Generation

The final component of the framework leverages modern large language models (LLMs) to automatically generate executable automation scripts from the specifications created in previous stages. This capability represents a significant advancement in web automation, enabling the transformation of structured specifications into functional code without human intervention.

Multiple state-of-the-art AI models were evaluated for this purpose, including OpenAI's GPT-4o-mini and GPT-3o-mini, Anthropic's Claude 3.5 Haiku, Claude 3.5 Sonnet, and Claude 3.7 Sonnet, as well as Google's Gemini Flash with thinking mode and Gemini Pro 2.5. While performance varied across models, all demonstrated the ability to generate working scripts with varying degrees of accuracy and completeness.

An interesting finding is that although the framework includes carefully crafted prompts to guide script generation, even relatively simple prompts often yielded correct scripts. This suggests that modern LLMs possess substantial knowledge about HTTP interactions, web automation, and Python programming paradigms relevant to web scraping and form submission.

It's important to note that while script generation is a valuable outcome of this framework, the broader significance lies in demonstrating that machine-readable specifications can enable AI agents to autonomously interact with web applications. This points toward a future where websites may include dedicated sections with specifications intended not for human users but for AI agents. Such specifications would allow these agents to automatically complete tasks without navigating through traditional user interfaces, significantly improving efficiency and reducing the need for manual intervention in repetitive workflows.

3.6.1 LLM Integration Architecture

The framework implements a modular architecture for AI integration, allowing different LLM providers to be used interchangeably (see Figure 12). The core of this architecture is an abstract base class (AIScriptGenerator) that defines a

common interface, with provider-specific implementations for different AI services.

```
AIScriptGenerator (abstract)
  ↓
  ├── ClaudeScriptGenerator
  └── OpenAIScriptGenerator
```

Figure 12: Abstract Class for AI Script Generator

This design follows the Strategy pattern, allowing the framework to switch between AI providers without changing the core logic. A factory function (get_script_generator) returns the appropriate implementation based on user selection, providing a clean abstraction layer between the framework and external AI services.

3.6.2 Sophisticated Prompt Engineering

The effectiveness of AI-generated scripts relies heavily on sophisticated prompt engineering. The framework implements a carefully crafted prompt template:

I need Python code for a web automation script based on the following specification.

The specification contains all the details about requests, parameters, and data flow.

The goal is to create a robust script that exactly follows this specification, including proper error handling, parameter extraction, and session management.

Please generate complete, production-ready code that implements all aspects of this specification. Include all necessary imports, functions for parameter extraction, and proper HTTP request handling.

The code should manage cookies and session state correctly.

Only output the full Python code with no explanations before or after.

Ensure all details from the specification are accurately implemented in the code.

This prompt design incorporates several advanced techniques:

1. **Clear Task Definition**: Explicitly states the required output format (Python code)
2. **Context Setting**: Establishes the automation context and requirements
3. **Quality Expectations**: Sets expectations for production-ready, robust code
4. **Output Constraints**: Specifies that only code should be returned, no explanations
5. **Content Boundary**: Clearly delineates where the specification begins

The prompt is deliberately crafted to guide the AI toward generating complete, functional code rather than explanations or partial implementations. This focus on concrete implementation helps overcome the tendency of some models to provide educational content rather than practical solutions.

3.6.3 Future Extensions

The AI-based script generation component is built with scalability in mind, allowing for several future enhancements that aim to strengthen its robustness and adaptability. One key direction is the integration of a feedback loop mechanism, enabling the system to test generated scripts and iteratively improve their quality based on execution results. Additionally, by combining outputs from multiple language models in a multi-model ensemble approach, the framework can leverage diverse strengths from different AI systems to produce more reliable automation. Incremental refinement techniques can further enhance scripts through step-by-step feedback-driven improvements. Domain-specific fine-tuning is also envisioned to tailor model behavior for particular applications, boosting accuracy and contextual understanding. To fully realize agent-driven automation, the development of specification formats specifically designed for AI agents is a crucial step—formats that prioritize machine readability and structured interaction semantics over human-focused representations. Altogether, these enhancements support a future where enterprise applications are interfaced through intelligent agents that consume specifications directly, bypassing traditional UI layers and ushering in a new model of human-AI collaboration in enterprise automation.

Chapter 4 Results and Discussion

This chapter presents the evaluation of the proposed reverse engineering framework through practical application to real-world web systems. Rather than relying on abstract quantitative metrics for a novel framework, this research adopts a case study-based approach focused on practical applicability. The evaluation assesses whether the framework can effectively analyze complex web applications with diverse architectures, identify data dependencies between HTTP requests, generate accurate specifications reflecting actual application behavior, and produce functional automation scripts capable of replicating user interactions. Due to information security considerations, publicly accessible web applications were selected for testing instead of internal enterprise systems, representing varied domains including education, transportation, recreation, and government services.

4.1 Case Study Implementation

Four distinct web applications were selected to evaluate the framework's capabilities across different technology stacks and complexity levels:

- **Moodle Learning Management System** (moodle2.ntust.edu.tw, evaluated in April 2025) - Academic portal
- **Taiwan Railway Reservation** (tip.railway.gov.tw, evaluated in April 2025) - Transportation service
- **Wuling Farm Camping Reservation** (camping.wuling-farm.com.tw, evaluated in April 2025) - Reservation system

- **Motor Vehicle Driver Information Service** (mvdis.gov.tw, evaluated in April 2025) - Government service portal

The target functions and key metrics for each application are summarized in Table 4.1 below:

Table 3: Summary of Case Study Applications and Results

Application	**Target Function**	**Complexity**	**Nodes**	**Parameter Extraction Success**	**Specification Completeness**	**Script Functionality**
Moodle LMS	Login & retrieve course info	High	24 (3 relevant)	80%	High	100%
Taiwan Railway	Login & book tickets	Medium	5	100%	High	100%
Wuling Farm	Login & reserve room	Low	4	100%	High	100%
MVDIS	Check traffic violations	Low	2	100%	High	100%

For each application, the framework generated visual dependency graphs illustrating request relationships. These visualizations proved invaluable for understanding the underlying structure of each application, particularly in complex systems where manual reverse engineering would be time-consuming. The most dramatic example was the Moodle system, where the framework's analysis reduced

24 total requests (see Figure 13) to just 3 critical nodes (see Figures 14) required for the target functionality.

Key observations from the implementation phase:

- **Dependency mapping effectiveness**: The framework successfully identified causal relationships between requests, as visualized in the dependency graphs (see Figure 13 - 18)
- **Complexity reduction**: Particularly effective in high-complexity applications like Moodle, where unnecessary requests were filtered out (see Figure 13 - 14)
- **Unknown parameter identification**: The framework automatically flagged parameters that couldn't be directly mapped, listing them in user_input.json (see Figure 16)

Figure 13: moodle2.ntust.edu.tw: All Nodes

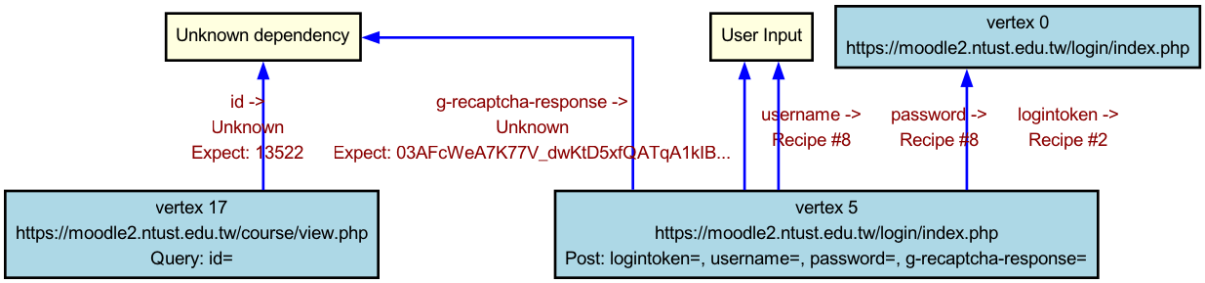

Figure 14: moodle2.ntust.edu.tw: Dependent Nodes Only

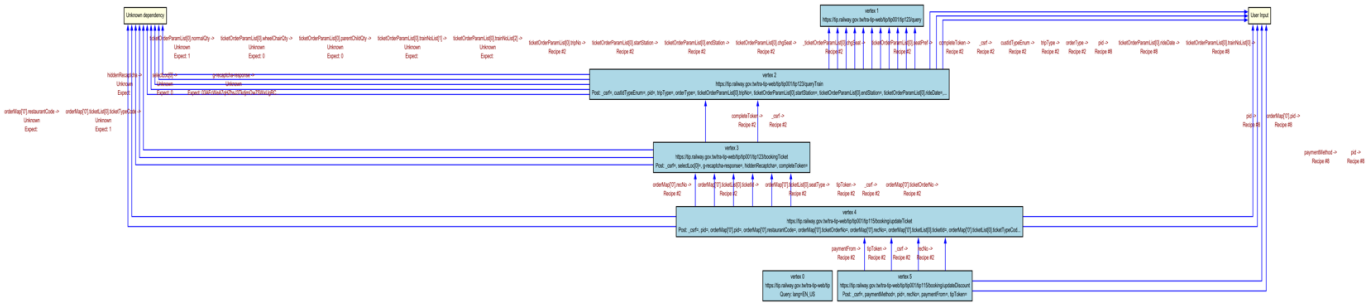

Figure 15: tip.railway.gov.tw: All Nodes


```
{
    "pid": "A1[redacted]",
    "ticketOrderParamList[0].rideDate": "2025/04/01",
    "ticketOrderParamList[0].trainNoList[0]": "149",
    "paymentMethod": "ONSITE",
    "unknown_posts": {
        "2": {
            "ticketOrderParamList[0].normalQty": "1",
            "ticketOrderParamList[0].wheelChairQty": "0",
            "ticketOrderParamList[0].parentChildQty": "0",
            "ticketOrderParamList[0].trainNoList[1]": "",
            "ticketOrderParamList[0].trainNoList[2]": ""
        },
        "3": {
            "selectLoc[0]": "0",
            "g-recaptcha-response": "03AFcWeA7qHZhsJ3TkdjmOw7SWxUgRCSBLZ33H
            "hiddenRecaptcha": ""
        },
        "4": {
            "orderMap['0'].restaurantCode": "",
            "orderMap['0'].ticketList[0].ticketTypeCode": "1"
        }
    }
}
```


Figure 16: tip.railway.gov.tw: User Input File (user_input.json)

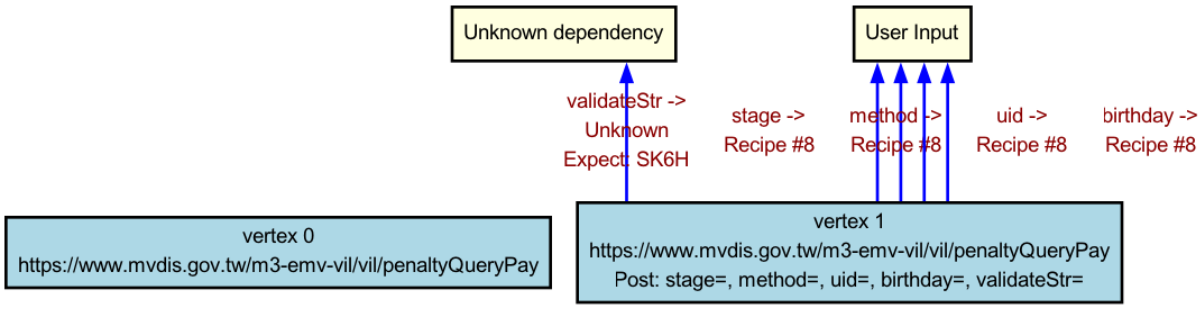

Figure 17: mvdis.gov.tw: All Nodes

Figure 18: camping.wuling-farm.com.tw: All Nodes

4.2 Analysis of Framework Performance

The framework demonstrated variable performance across different aspects of the reverse engineering process, with both strengths and limitations clearly identified.

Performance Strengths:

- High success rates for parameter extraction (80-100%)
- Complete specifications generated across all applications
- Fully functional scripts for all target workflows
- Efficient parameter flagging for manual verification

Challenging Parameter Types:

1. **Security mechanisms**: CAPTCHA responses (g-recaptcha-response)
2. **Dynamic values**: Client-side generated tokens and random numbers
3. **User-specific inputs**: Preferences and selections not captured during recording

Table 4.2 details the specific challenging parameters encountered in each application:

Table 4: Challenging Parameters Encountered by Application

Application	Challenging Parameter	Type	Resolution Method
Moodle LMS	g-recaptcha-response	CAPTCHA	Manual input/cookie import
Moodle LMS	id (course identifier)	Static value	Manual specification
Taiwan Railway	g-recaptcha-response	CAPTCHA	Manual input/cookie import
Taiwan Railway	ticket parameters	User preferences	User input file
Wuling Farm	recaptcha_response	CAPTCHA	Manual input/cookie import
MVDIS	validateStr	CAPTCHA	Manual input/cookie import

The framework's parameter identification capabilities were particularly valuable in complex workflows. For example, in the Taiwan Railway Reservation case, several ticket parameters were correctly identified as requiring user input despite not being directly traceable through HTML elements. These parameters were automatically added to the unknown_queries and unknown_posts sections in the user_input.json file, allowing for straightforward review and configuration.

The analysis revealed a correlation between application complexity and analysis time:

- **Low complexity** (2-4 nodes): Analysis completed in 2-3 minutes
- **Medium complexity** (5-10 nodes): Analysis completed in 4-6 minutes
- **High complexity** (10+ nodes): Analysis completed in 7-10 minutes

This scaling is primarily due to the causality analysis phase using HAR files. Once dependencies were established, both specification generation and script creation proceeded rapidly regardless of application complexity, demonstrating the efficiency of the framework's two-phase approach.

4.3 Integrated Evaluation and Implications

The proposed framework demonstrated consistent performance across both public web applications and internal enterprise systems, validating its adaptability in diverse environments. Evaluation of four target systems revealed high parameter extraction success rates (80–100%) and complete specification coverage, highlighting the effectiveness of the two-phase methodology—first establishing parameter causality from HAR file analysis, then rapidly generating specifications and executable scripts. Notably, the Moodle case illustrated the framework's ability to reduce workflow complexity, pinpointing only 3 out of 24 HTTP nodes as critical for task completion.

While generated specifications remained stable, the automation scripts produced by different AI models exhibited stylistic variation, yet consistently maintained functional correctness. This observation suggests strong generalizability and opens opportunities for further improvements through AI model refinement. The primary technical limitation identified was the framework's reduced capability in handling dynamic parameters generated entirely via client-side JavaScript, especially CAPTCHA fields. However, this constraint has limited impact in enterprise systems, where such mechanisms are uncommon.

Theoretically, the research affirms that reverse engineering application logic through network-level traffic is feasible and effective without access to source code. The recipe-based parameter classification system introduces a structured way to represent dynamic behaviors, bridging the gap between traditional manual testing and

automated AI-driven solutions. Practically, the framework provides valuable tools across multiple enterprise functions: IT departments can automate legacy systems; software maintenance teams can generate documentation and visualize data dependencies; QA engineers benefit from automatically derived test scripts; and security analysts can trace parameter flows to identify potential vulnerabilities.

Altogether, this work contributes a practical and extensible solution for enterprise web automation and lays the foundation for further AI integration into software comprehension and automation workflows.

4.4 Limitations and Future Research

Technical Limitations:

- Difficulty handling client-side JavaScript-generated parameters and security mechanisms
- Reliance on HAR files which may not capture all network activity
- Challenges with highly dynamic interfaces and complex navigation logic

Future Research Directions:

1. Enhanced JavaScript analysis to better understand dynamic parameter generation
2. Extended recipe system for complex extraction scenarios
3. Comprehensive validation on complex enterprise applications
4. Integration of security and compliance checks

Chapter 5 Conclusion

This research demonstrates the effectiveness of a novel framework that bridges the gap between web applications and AI automation through specification generation. A key innovation is the integration with large language models to transform these specifications into functional automation scripts, with tests across multiple AI platforms validating this approach. The consistent results across diverse case studies confirm the framework's ability to enable automation where direct API access is unavailable.

The framework points toward a future paradigm where applications include machine-readable specifications designed for AI consumption, representing an alternative integration pathway that doesn't require traditional APIs. Rather than retrofitting existing applications with new interfaces, this approach leverages their current behavior to enable automation.

As enterprises increasingly adopt AI for process automation, generating specifications from existing web applications will become a critical capability. This research provides both the conceptual foundation and practical implementation for such capabilities, contributing to the evolution of enterprise software integration by transforming implicit application knowledge into explicit specifications that serve as interfaces for AI agents.

References

[1] V. Felmetsger, L. Cavedon, C. Kruegel, and G. Vigna, "Toward Automated Detection of Logic Vulnerabilities in Web Applications," in Proc. USENIX Security Symposium, Washington, DC, USA, 2010, pp. 143–160.

[2] A. Mesbah, A. van Deursen, and S. Lenselink, "Crawling Ajax-based Web Applications through Dynamic Analysis of User Interface State Changes," ACM Trans. Web, vol. 6, no. 1, pp. 1–30, 2012.

[3] HAR Working Group, "HTTP Archive (HAR) Format," W3C Community Group, 2012. [Online]. Available: https://w3c.github.io/web-performance/specs/HAR/

[4] C. S. Păsăreanu, R. Pelánek, J. R. Kiniry, and W. Visser, "Combining Symbolic and Concrete Execution for Data Structure Analysis," in Proc. ISSTA, Toronto, ON, Canada, 2011, pp. 37–48.

[5] A. Hogue, "Web Scraping with Regular Expressions and XPath," O'Reilly Web 2.0 Expo, 2008.

[6] Y. Minamide, "Static Approximation of Dynamically Generated Web Pages," in Proc. 14th International Conference on World Wide Web (WWW), Chiba, Japan, 2005, pp. 432–441.

[7] B. Livshits and M. S. Lam, "Finding Security Vulnerabilities in Java Applications with Static Analysis," in Proc. USENIX Security Symposium, Baltimore, MD, USA, 2005, pp. 271–286.

[8] D. S. Linthicum, Enterprise Application Integration, Boston, MA, USA: Addison-Wesley, 2000.

[9] W. M. P. van der Aalst, M. Bichler, and A. Heinzl, “Robotic Process Automation,” Bus. Inf. Syst. Eng., vol. 60, no. 4, pp. 269–272, 2018.

[10] UiPath Inc., “UiPath Platform Overview,” 2023. [Online]. Available: https://www.uipath.com/

[11] W3C, “Web Services Description Language (WSDL) 1.1,” Mar. 2001. [Online]. Available: https://www.w3.org/TR/wsdl

[12] OpenAPI Initiative, “OpenAPI Specification Version 3.1.0,” 2023. [Online]. Available: https://swagger.io/specification/

[13] SmartBear Software, “Gherkin Language Syntax,” 2020. [Online]. Available: https://cucumber.io/docs/gherkin/

[14] M. Chen et al., “Evaluating Large Language Models Trained on Code,” arXiv preprint, arXiv:2107.03374, Jul. 2021.

[15] B. Rozière et al., “Code Llama: Open Foundation Models for Code,” arXiv preprint, arXiv:2308.12950, Aug. 2023.

[16] Google DeepMind, “Gemini Models Overview,” 2024. [Online]. Available: https://deepmind.google/

[17] S. Yao, D. Zhao, D. Yu, and Q. Liu, “ReAct: Synergizing Reasoning and Acting in Language Models,” arXiv preprint, arXiv:2210.03629, Oct. 2022.

[18] T. Schick, L. R. Kaiser, and H. Schütze, “Toolformer: Language Models Can Teach Themselves to Use Tools,” arXiv preprint, arXiv:2302.04761, Feb. 2023.

[19] TestRigor Inc., “TestRigor Documentation,” 2023. [Online]. Available: https://testrigor.com/

[20] Selenium Project, “Selenium IDE,” 2023. [Online]. Available: https://www.selenium.dev/selenium-ide/

[21] Puppeteer Team, “Puppeteer: Headless Chrome Node.js API,” GitHub, 2023. [Online]. Available: https://github.com/puppeteer/puppeteer

[22] Checkly, “Headless Recorder,” Chrome Extension, 2022. [Online]. Available: https://checklyhq.com/headless-recorder

[23] J. D. Zettlemoyer et al., “Multimodal Chain-of-Thought Reasoning in Language Models,” arXiv preprint, arXiv:2205.09712, May 2022.

[24] LangChain, “LangChain: Framework for Developing Applications Powered by Language Models,” 2023. [Online]. Available: https://www.langchain.com/

[25] Anthropic, “Claude 3 Model Family Overview,” 2024. [Online]. Available: https://www.anthropic.com/news/claude-3-family